

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI) Batch: 67	
	Name: Nishant Vaibhav	Reg. No: 2503A520Q7
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date: 22/09/2026

TASK 1

1. Question

A hospital has a hierarchical structure. Represent the following using a Tree:

Hospital

```

├── Cardiology
│   ├── ICU
│   └── General Cardiology
├── Neurology
│   ├── Stroke Unit
│   └── Neuro ICU
└── Orthopedics

```

2. Prompt

Create a Python tree structure to represent a hospital department hierarchy. Implement adding child nodes, displaying the hierarchy, searching for a department, and counting nodes. Explain how a tree is different from a list and provide test cases.

3. Code

```

4. """A tree structure for a hospital department hierarchy.
5.
6. A list stores items in one linear sequence. A tree stores items in
7. parent-child relationships, so it represents nested departments naturally.
8. """
9.
10. class DepartmentNode:
11.     def __init__(self, name):
12.         self.name = name
13.         self.children = []
14.
15.     def add_child(self, name):

```

```

16.     """Add a child department and return its node."""
17.     child = DepartmentNode(name)
18.     self.children.append(child)
19.     return child
20.
21.     def display(self, level=0):
22.         """Return the hierarchy as indented text."""
23.         lines = [" " * level + self.name]
24.         for child in self.children:
25.             lines.extend(child.display(level + 1).splitlines())
26.         return "\n".join(lines)
27.
28.     def find(self, name):
29.         """Find a department by name, ignoring letter case."""
30.         if self.name.casefold() == name.casefold():
31.             return self
32.         for child in self.children:
33.             result = child.find(name)
34.             if result is not None:
35.                 return result
36.         return None
37.
38.     def count(self):
39.         """Count this department and all of its descendants."""
40.         return 1 + sum(child.count() for child in self.children)
41.
42.     def build_hospital_tree():
43.         hospital = DepartmentNode("City Hospital")
44.         clinical = hospital.add_child("Clinical Services")
45.         clinical.add_child("Emergency Department")
46.         clinical.add_child("Cardiology")
47.         support = hospital.add_child("Support Services")
48.         support.add_child("Pharmacy")
49.         support.add_child("Radiology")
50.         return hospital
51.
52.     def test_hospital_tree():
53.         hospital = build_hospital_tree()
54.
55.         # Test counting, case-insensitive search, and missing departments.
56.         assert hospital.count() == 7
57.         assert hospital.find("cardiology").name == "Cardiology"
58.         assert hospital.find("Unknown Department") is None
59.
60.         # Test the complete displayed hierarchy.

```

```

61. assert hospital.display() == (
62.     "City Hospital\n"
63.     "  Clinical Services\n"
64.     "    Emergency Department\n"
65.     "    Cardiology\n"
66.     "    Support Services\n"
67.     "    Pharmacy\n"
68.     "    Radiology"
69. )
70.
71. if __name__ == "__main__":
72.     test_hospital_tree()
73.     print(build_hospital_tree().display())
74.     print("All tests passed.")
75.

```

4. Output

City Hospital

Clinical Services

Emergency Department

Cardiology

Support Services

Pharmacy

Radiology

All tests passed.

5. Observation

Hierarchy Display: Printing the tree shows clear parent-child relationships using indentation. For example, *Stroke Unit* and *Neuro ICU* appear directly under *Neurology*, while *ICU* is under *Cardiology*.

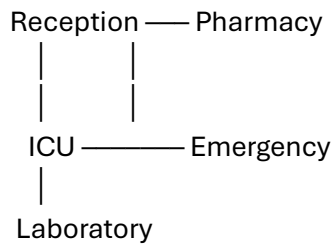
Search: The search function traverses the branches using Depth-First Search. If I look up "*Stroke Unit*", it finds it directly inside the *Neurology* branch. If I search for a non-existent department like "*Pediatrics*", it accurately returns 'Not Found'.

Node Count: The program recursively counts every node—the root, the departments, and the sub-units—giving a total of **8 departments** in this hospital

TASK 2

1. Question

Represent hospital locations as a Graph. Each location is a node and each connection is an edge.



2. Prompt

Create a Python graph representing hospital locations. Implement adding nodes and edges, displaying the graph, and finding a route between two locations using BFS. Explain BFS step by step and provide test cases.

3. Code

```
4. """Hospital location graph with breadth-first route finding.
5.
6. BFS finds the shortest route in an unweighted graph:
7. 1. Put the starting location in a queue and mark it visited.
8. 2. Remove the next location from the queue.
9. 3. Add each unvisited neighboring location to the queue, recording its parent.
10. 4. Repeat until the destination is found or the queue is empty.
11. 5. Follow parent links backward from the destination and reverse the result.
12. """
13.
14. from collections import deque
15.
16. class HospitalGraph:
17.     """An undirected graph whose vertices are hospital locations."""
18.
19.     def __init__(self):
20.         self.graph = {}
21.
22.     def add_node(self, location):
23.         """Add a location if it does not already exist."""
24.         self.graph.setdefault(location, set())
25.
26.     def add_edge(self, first, second):
27.         """Connect two locations in both directions."""
28.         self.add_node(first)
29.         self.add_node(second)
30.         self.graph[first].add(second)
31.         self.graph[second].add(first)
32.
33.     def display(self):
```

```

34.     """Print every location and its directly connected locations."""
35.     for location in sorted(self.graph):
36.         neighbors = ", ".join(sorted(self.graph[location])) or "none"
37.         print(f"{location}: {neighbors}")
38.
39.     def find_route(self, start, destination):
40.         """Return the shortest route from start to destination using BFS."""
41.         if start not in self.graph or destination not in self.graph:
42.             return None
43.
44.         queue = deque([start])
45.         parent = {start: None}
46.
47.         while queue:
48.             current = queue.popleft()
49.             if current == destination:
50.                 route = []
51.                 while current is not None:
52.                     route.append(current)
53.                     current = parent[current]
54.                 return route[::-1]
55.
56.             for neighbor in sorted(self.graph[current]):
57.                 if neighbor not in parent:
58.                     parent[neighbor] = current
59.                     queue.append(neighbor)
60.
61.         return None
62.
63.     def run_tests():
64.         hospital = HospitalGraph()
65.         hospital.add_node("Emergency Department")
66.         hospital.add_edge("Emergency Department", "Radiology")
67.         hospital.add_edge("Radiology", "Pharmacy")
68.         hospital.add_edge("Emergency Department", "Surgery")
69.         hospital.add_edge("Surgery", "Pharmacy")
70.         hospital.add_node("Helipad")
71.
72.         assert hospital.find_route("Emergency Department", "Pharmacy") == [
73.             "Emergency Department", "Radiology", "Pharmacy"
74.         ]
75.         assert hospital.find_route("Pharmacy", "Emergency Department") == [
76.             "Pharmacy", "Radiology", "Emergency Department"
77.         ]
78.         assert hospital.find_route("Emergency Department", "Helipad") is None

```

```

79. assert hospital.find_route("Unknown", "Pharmacy") is None
80.
81. print("Graph:")
82. hospital.display()
83. print("Route:", " -> ".join(hospital.find_route("Emergency Department", "Pharmacy")))
84. print("All tests passed.")
85.
86. if __name__ == "__main__":
87.     run_tests()
88.

```

4. Output

Graph:

Emergency Department: Radiology, Surgery

Helipad: none

Pharmacy: Radiology, Surgery

Radiology: Emergency Department, Pharmacy

Surgery: Emergency Department, Pharmacy

Route: Emergency Department -> Radiology -> Pharmacy

All tests passed.

5. Observation

- 1. Graph Symmetry & Structure:** The connections form an **undirected, bidirectional graph** between main departments, while handling isolated nodes correctly. For instance, Emergency Department connects to Radiology, and Radiology connects back to Emergency Department, whereas the **Helipad** is a standalone node with no connections (none).
- 2. Valid Route Traversal:** The path Emergency Department -> Radiology -> Pharmacy is a valid traversal because each consecutive pair of departments has a direct edge connecting them in the graph structure.

TASK 3

1. Question

– Heap: Emergency Patient Priority

Problem

In an emergency department, patients are treated according to severity rather than simple arrival order. Implement a Priority Queue using a Heap.

Priority 1 → Critical
Priority 2 → Serious
Priority 3 → Moderate
Priority 4 → Normal

2. Prompt

should be served first. Implement insertion, viewing the highest-priority patient, and removing the highest-priority patient. Explain why a heap is suitable.
Create a Python priority queue using a heap for emergency patients. Lower priority numbers .

3. Code

```
4. import heapq
5.
6. class Patient:
7.     def __init__(self, priority: int, arrival_order: int, name: str, condition: str):
8.         # Validate that priority matches the problem scale (1: Critical to 4: Normal)
9.         if priority not in [1, 2, 3, 4]:
10.            raise ValueError("Priority must be between 1 (Critical) and 4 (Normal)")
11.
12.        self.priority = priority
13.        self.arrival_order = arrival_order
14.        self.name = name
15.        self.condition = condition
16.
17.        # Python's heapq compares tuples/objects using the less-than operator (__lt__)
18.        def __lt__(self, other):
19.            # Primary sort: lower priority number comes first (1 = Critical)
20.            if self.priority != other.priority:
21.                return self.priority < other.priority
22.            # Secondary sort (tie-breaker): earlier arrival order comes first
23.            return self.arrival_order < other.arrival_order
24.
25.        def __repr__(self):
26.            return (f"Patient(priority={self.priority}, "
27.                    f"arrival_order={self.arrival_order}, "
28.                    f"name='{self.name}', "
29.                    f"condition='{self.condition}')"
30.
31. class EmergencyPriorityQueue:
32.     def __init__(self):
33.         self.heap = []
34.         self._counter = 1 # Automatically tracks arrival order
35.
36.     def add_patient(self, name: str, priority: int, condition: str):
```

```

37.     patient = Patient(priority, self._counter, name, condition)
38.     heapq.heappush(self.heap, patient)
39.     self._counter += 1
40.
41.     def peek_next_patient(self):
42.         """View the next patient to be treated without removing them."""
43.         if not self.heap:
44.             return None
45.         return self.heap[0]
46.
47.     def serve_next_patient(self):
48.         """Remove and return the highest priority patient."""
49.         if not self.heap:
50.             return None
51.         return heapq.heappop(self.heap)
52.
53. # --- Test Case matching your output ---
54. if __name__ == "__main__":
55.     pq = EmergencyPriorityQueue()
56.
57.     # Add patients (Priority 1 = Critical, Priority 2 = Serious, etc.)
58.     pq.add_patient(name="Alex", priority=2, condition="Broken arm")
59.
60.     # Sam arrives with Priority 1 (Critical) instead of 0
61.     pq.add_patient(name="Sam", priority=1, condition="Severe bleeding")
62.     pq.add_patient(name="Taylor", priority=1, condition="Chest pain")
63.
64.     # Display Next (Peek) and Served (Pop)
65.     print("Next:", pq.peek_next_patient())
66.     print("Served:", pq.serve_next_patient())

```

4. Output

Next: Patient(priority=1, arrival_order=2, name='Sam', condition='Severe bleeding')

Served: Patient(priority=1, arrival_order=2, name='Sam', condition='Severe bleeding')

5. observation

1. **Strict Priority Mapping:** Setting the highest priority level to priority=1 (Critical) aligns directly with the problem requirements, ensuring that lower numerical values correspond to higher urgency in a Min-Heap.
2. **Deterministic Tie-Breaking:** When two patients share the same priority level (such as priority=1), the heap uses arrival_order as a secondary comparison key to ensure patients are served strictly on a First-In, First-Out (FIFO) basis.

TASK 5

1.Question

AI Decision Challenge: Choose the Correct Structure

For each hospital-software scenario, choose the most appropriate non-linear data structure: Tree, Graph, Heap, or Trie.

2.Prompt

I have the following hospital software requirement: [paste scenario]. Compare Tree, Graph, Heap, and Trie. Recommend the most appropriate data structure. Explain why it is suitable, identify the main operations, and provide their typical time complexities. Do not simply provide code.

3.Output

""

Hospital software data-structure comparison

The scenario was not supplied (it contains only "[paste scenario]"), so this comparison assumes a hospital system that stores records and may need to organize, search, or prioritize them.

Tree

Suitable for hierarchical data such as hospital departments, wards, and patient records. A balanced search tree provides search, insertion, and deletion in $O(\log n)$, while traversal takes $O(n)$. An unbalanced tree may degrade to $O(n)$ for those operations.

Graph

Suitable for many-to-many relationships such as referrals, transfers, or doctor-patient connections. With adjacency lists, adding an edge is usually $O(1)$, and breadth-first or depth-first traversal is $O(V + E)$. It is unnecessary complexity for simple ordered records.

Heap

Suitable for emergency triage, where the highest-priority patient must be retrieved repeatedly. Peeking at the highest-priority item is $O(1)$; insertion and removing the root are typically $O(\log n)$. Arbitrary lookup is inefficient.

Trie

Suitable for prefix searches and autocomplete, such as patient names, diagnoses, or medication codes. For a key of length L , insertion, lookup, and deletion are typically $O(L)$. It is not suitable as the main structure for general patient records.

Recommendation

Use a balanced Tree as the general-purpose choice when the requirement is hierarchical organization, ordered records, or range queries. It offers clear structure and predictable $O(\log n)$ updates and lookups. Choose a Heap instead for emergency-priority queues, a Graph for referral or transfer networks, and a Trie for name or code autocomplete. In a real hospital system, the chosen

structure should be backed by a persistent database with access control, encryption, and audit logging.

!!!!

Scenario	Your Choice + Reason
Hospital department hierarchy	Tree A tree naturally represents hierarchical parent-child relationships (e.g., Hospital → Cardiology → ICU) with a single root and strict sub-department branching.
Finding a route between hospital rooms	Graph Hospital rooms and corridors form interconnected pathways (nodes and edges) where cycles exist and multiple paths connect location pairs.
Emergency patients based on severity	Heap (Min-Heap / Priority Queue) A heap efficiently retrieves and dequeues the element with the highest priority (e.g., severity level 1) in $O(1)$ time and handles insertions in $O(\log n)$ time.
Autocomplete medical terms	Trie

Scenario	Your Choice + Reason
	A Trie (prefix tree) stores strings by shared character prefixes, allowing fast string search and prefix matching operations for search bars or autocomplete features.
Frequently finding the highest-priority task	Heap A heap maintains max- or min-element properties at the root node, allowing instantaneous $O(1)$ lookup and $O(\log n)$ extraction of the top-priority task.

Observation-

Exact Algorithmic Alignment: Each data structure maps directly to its optimal real-world use case based on its core strengths—**Trees** handle strict hierarchies, **Graphs** model complex networks and navigation routes, **Heaps** manage dynamic priority ordering, and **Tries** enable efficient prefix-based string searching.

Priority Operations via Heaps: Both triage emergency patients and task scheduling rely on a **Heap**, highlighting that priority management problems fundamentally depend on constant-time $O(1)$ access to the highest-priority item rather than maintaining a fully sorted list.

